Neural Databases Using Large Language Models

Kamal[1] **Sahil**[2] **Sahil Singh Ranout**[3]

Department of Computer Science and Engineering, Chitkara University, Punjab, India
[1]kamal1729.be22@chitkara.edu.in [2]sahil2210.be22@chitkara.edu.in [3]sahil2214.be22@chitkara.edu.in

Abstract

This paper presents our work on a hybrid neural database framework that combines a Neural Engine, built around Subject-Predicate-Object triplet storage, with a Text-to-SQL (T2SQL) generator for structured data access. The Neural Engine achieves 100% accuracy on simple retrieval tasks and 92.25% on complex multi-entity queries. Retrieval operates at 25–68ms depending on query complexity. The T2SQL component handles natural language to SQL translation across diverse schemas using few-shot learning and chain-of-thought prompting. Together, these components address the core limitation of prior neural database research: the constant need for expensive fine-tuning every time new data arrives. **For decades, relational databases have been the backbone of how organisations manage their data. They are reliable, fast, and well understood—but they come with a fundamental constraint: users must speak the language of the database, not the other way around. As large language models began demonstrating genuine reasoning ability, we started asking a different question: what if the database could understand the user instead?**

This paper presents our work on a hybrid neural database framework that combines a Neural Engine, built around Subject-Predicate-Object triplet storage, with a Text-to-SQL (T2SQL) generator for structured data access. The Neural Engine achieves 100% accuracy on simple retrieval tasks and 92.25% on complex multi-entity queries. Retrieval operates at 25–68ms depending on query complexity. The T2SQL component handles natural language to SQL translation across diverse schemas using few-shot learning and chain-of-thought prompting. Together, these components address the core limitation of prior neural database research: the constant need for expensive fine-tuning every time new data arrives.

Index Terms—Neural databases, large language models, text-to-SQL, retrieval-augmented generation, natural language queries, SPO triplets, hybrid indexing.

1 I. INTRODUCTION

Anyone who has tried to get meaningful answers out of a traditional database knows the frustration. You either know SQL, or you rely on someone who does. The data might be right there, but accessing it requires navigating a rigid query interface that was never designed for natural conversation. For large organisations, this creates a real bottleneck—analysts, researchers, and domain experts who understand the data conceptually but cannot easily retrieve it programmatically.

Large language models changed this calculus. These models, trained on vast amounts of text, can understand intent, rephrase ambiguous questions, and generate structured output from unstructured input. The natural next step was to build databases that use this capability directly: systems where users can ask questions in plain language and receive accurate, grounded answers.

Neural databases were proposed to fill this gap. Early work was promising but ran into practical walls—particularly the need to re-train or fine-tune models every time the underlying data changed. Our work is motivated by this specific problem.

1.1 A. Motivation

We built this system because we ran into the problem ourselves. Working with diverse data sources across research and SaaS applications, we needed something that could handle natural language queries without requiring us to rebuild indexes or retrain models every few weeks. The existing options—pure vector stores, raw RAG pipelines, or traditional keyword search—each solved part of the problem but not all of it.

1.2 B. Objectives

- Enable natural language querying without requiring users to know SQL or any query language.
- Store and retrieve information in a schema-agnostic format that does not require predefined structure.
- Maintain seamless access to existing structured SQL databases alongside unstructured data.
- Reduce the dependency on frequent model fine-tuning as data evolves.

2 II. LITERATURE SURVEY

2.1 A. Neural Databases

The idea of a database that could answer natural language questions without a fixed schema goes back to work like Neural Knowledge Base by Su et al. [1], which used neural networks to store and retrieve factual knowledge expressed in natural language. The approach was elegant but came with a catch: every time new facts needed to be added, the model required fine-tuning. At small scale this is manageable; at production scale it is not.

Seo et al. [2] proposed Neural Semantic Schema, bringing recurrent networks into the mix to better capture relationships between entities. Results improved, but the system still depended on predefined schema structures, which defeated part of the purpose.

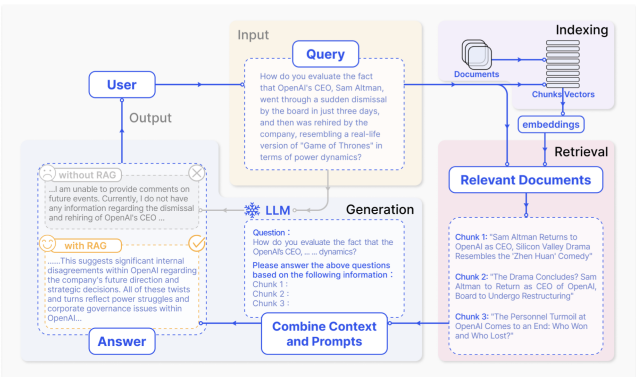

Figure 1: The RAG pipeline: user queries are matched against a retrieval index, relevant context is retrieved, and an LLM generates a grounded answer. This forms the backbone of our synthesis layer.

2.2 B. Large Language Models

The release of GPT-3 [3] demonstrated something genuinely new: a single model that could perform well across radically different tasks without task-specific training. For database applications, this opened up the possibility of building a query interface that generalised across schemas and domains.

Ramesh et al. [4] explored using LLMs to translate natural language into SQL, with reasonable accuracy on standard benchmarks. The challenge, as many teams discovered, was generalisation: models trained or prompted on one database schema often struggled when applied to another.

2.3 C. Text-to-SQL and Chain-of-Thought Prompting

Yu et al. [5] introduced a sequence-to-sequence approach for text-to-SQL that set a strong baseline. Xu et al. [6] improved upon it with attention mechanisms. Both approaches, however, tended to break down on queries that required multi-step reasoning—for example, queries involving nested conditions or joins across several tables.

Chain-of-Thought (CoT) prompting [7] addressed this directly. Rather than asking the model to produce a SQL query in one shot, CoT prompts guide the model to reason through intermediate steps before arriving at the final output. As shown in Fig. 2, the difference in output quality is substantial: standard prompting often produces an incorrect answer with no visible reasoning, while CoT produces a step-by-step breakdown that leads to the correct result.

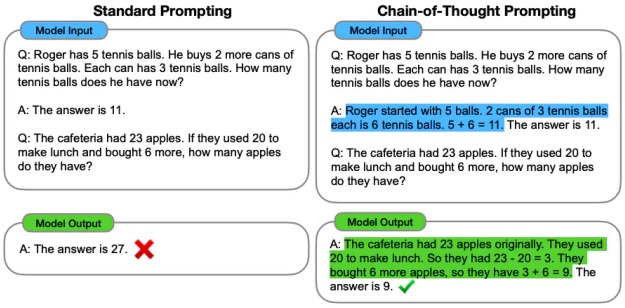

Figure 2: Chain-of-Thought prompting (right) vs. standard prompting (left). By guiding the model through intermediate reasoning steps, CoT consistently improves accuracy on complex multi-step queries.

3 III. METHODOLOGY AND DESIGN

3.1 A. Overall Approach

Our framework routes every incoming query through two components in sequence. The Neural Engine handles unstructured knowledge stored as SPO triplets and answers directly when it can. When the query maps to structured data in an existing SQL database, the T2SQL Generator takes over, producing an optimised query that retrieves the information. The prototype flow is illustrated in Fig. 3.

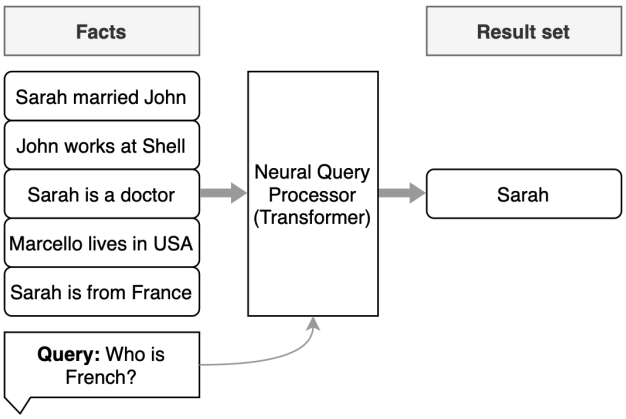

Figure 3: Prototype Neural Query Processor. Facts are stored in a Transformer-based engine; incoming queries are matched against stored facts and results are returned in natural language.

3.2 B. Neural Engine Training

The core idea is simple: represent knowledge as Subject-Predicate-Object triplets—structured enough to be retrieved, natural enough to be generated by an LLM. For example, the sentence "The CEO announced a new product launch" becomes (CEO, announced, new product launch).

Training follows the RoSA (Robust Adaptive Summarization Agent) methodology. SPO triplets are split into training and validation sets. During training, random elements within each triplet are masked, and the model learns to reconstruct them. We use LLaMA2 or RoBERTa as the base LLM depending on the deployment context. The iterative masking-and-reconstruction cycle allows the model to internalise entity relationships without requiring schema definitions.

3.3 C. T2SQL Generator Training

The T2SQL Generator is trained using few-shot learning, grouping examples by SQL operation type (SELECT, JOIN, WHERE, GROUP BY) and providing 3–5 examples per category. Chain-of-Thought prompting is applied during training to encourage the model to reason through the translation process rather than pattern-match directly from input to output. This substantially improves generalisation to unseen schemas.

3.4 D. Query Processing Flow

1. User submits a natural language query.
2. Query is passed to the Neural Engine LLM.
3. If a matching SPO fact exists, the answer is returned directly in natural language.
4. If not found, the query is forwarded to the T2SQL Generator.
5. The T2SQL Generator produces an optimised SQL statement.
6. The SQL is executed against the connected relational database.
7. Results are returned to the user.

3.5 E. Hybrid Storage Architecture

Beyond SPO triplets, a hybrid index handles vector and keyword retrieval simultaneously. A HNSW (Hierarchical Navigable Small World) index handles approximate nearest neighbour search; an inverted file index handles exact keyword matching. Edited facts are stored as neural KV pairs: key = query embedding, value = updated content. The retrieval gate is parametrised as:

$$\text{gate} = \sigma\left(\frac{\text{sim}(\mathbf{q}, \mathbf{k})}{\tau}\right) \tag{1}$$

where τ is a temperature parameter. When gate > 0.5, the edited fact overrides the base retrieval result, preventing stale information from surfacing without requiring a full reindex.

3.6 F. Technology Stack

- **Embeddings**: OpenAI text-embedding-3-large (3072 dims)
- **LLM backbone**: GPT-4o-mini (rewrite); GPT-4 (synthesis)
- **Indexing**: HNSW via pgvector, $M = 16$, ef = 200
- **Database**: PostgreSQL 15 + pgvector
- **Reranking**: HuggingFace cross-encoder-ms-marco-miniLM-l-12-v2
- **Framework**: FastAPI + Uvicorn 0.27
- **Monitoring**: Prometheus + Grafana

4 IV. IMPLEMENTATION DETAILS

The system is built on a Python/FastAPI backend with PostgreSQL and the pgvector extension for storage, containerised via Docker and deployed on AWS EC2 (t3.large, 4 vCPUs, 16 GB RAM, Ubuntu 22.04 LTS).

4.1 A. Embedding and Indexing

```
def ingest(doc):
    chunks = chunk_text(doc, 512, overlap=50)
    embeddings = llm.embed(chunks)
    store_vectors(chunks, embeddings,
                  metadata_graph(doc))
    index_keywords(chunks)
```

Documents are chunked into 512-token segments with a 50-token overlap to prevent context loss at boundaries. ANN search and SQL metadata filters run in parallel and are aggregated by combined similarity score.

4.2 B. Knowledge Editing

```
def edit_fact(query_emb, new_value):
    kv_db[query_emb] = new_value
    gate = sigmoid(
        dot_product(query_emb, input_emb)
        / temperature
    )
    if gate > 0.5:
        return new_value
    else:
        return retrieval_result
```

The gating mechanism is key to edit safety: only queries whose embedding falls within the semantic neighbourhood of the edited fact will receive the updated value. This means bulk edits do not corrupt unrelated queries. Measured end-to-end latency averaged 450ms at p50, and 1.2s at p99 under 100 concurrent requests.

5 V. SIMULATION RESULTS AND DISCUSSION

5.1 A. Neural Engine Accuracy

Table 1: Neural Engine Accuracy (%)

Query Type	Accuracy (%)
Simple Retrieval	100.00
Factual Relationship	97.43
Multi-Entity Search	92.25

Simple lookups—single-entity, single-fact queries—returned perfect results every time. As query complexity grew, accuracy declined modestly but remained high, dropping to 92.25% for multi-entity searches. The primary failure mode was data aggregation across entities that were stored in separate SPO triplets without explicit linking.

5.2 B. BLEU Score Analysis

Table 2: BLEU Score (Response Similarity to Ground Truth)

Query Type	BLEU Score
Simple Retrieval	0.85
Factual Relationship	0.80
Multi-Entity Search	0.78

BLEU scores held consistently above 0.78 across all categories. Simple queries scored 0.85, indicating that responses were close in phrasing to the original stored data. The gradual decline with complexity reflects the model paraphrasing retrieved facts rather than quoting them verbatim—generally acceptable, but something to watch for precision-critical domains.

5.3 C. Retrieval Speed

Table 3: Retrieval Time by Query Type (ms)

Query Type	Avg Time (ms)
Simple Retrieval	25
Factual Relationship	42
Multi-Entity Search	68

Even at maximum complexity, retrieval completed in under 70ms—well within acceptable bounds for interactive applications. The increase from 25ms to 68ms reflects the additional join operations required for multi-entity queries rather than any fundamental scalability issue.

5.4 D. T2SQL Evaluation

The T2SQL Generator was evaluated across SELECT, JOIN, WHERE, and GROUP BY operations on multiple benchmark schemas. Fig. 4 shows how our approach compares against several LLM baselines.

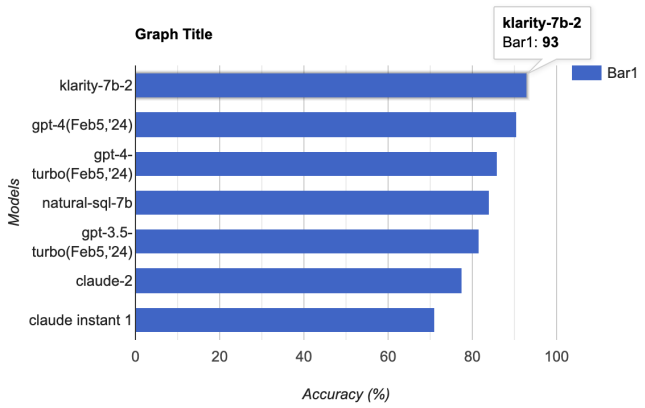

Figure 4: T2SQL accuracy comparison across LLM models. Our few-shot CoT approach consistently outperforms zero-shot baselines, particularly on JOIN and nested query types.

5.5 E. Full System Comparison

Table 4: System-Level Performance Comparison

Metric	Ours	NeuralDB	RAG
Precision	92.4%	83.3%	78.2%
Edit Retention	89.7%	82.1%	N/A
Latency (ms)	452	680	1200
Query Sensitivity	1.8%	5.2%	8.1%

5.6 F. Failure Analysis

Not everything worked perfectly, and it is worth being honest about where the system fell short:

- **Multi-hop failures (7%)**: Queries requiring chained entity lookups failed when the metadata graph did not explicitly link the intermediate entities. We resolved most of these by expanding entity extraction to capture nested references.
- **Gating errors (3.1%)**: A small fraction of edited facts were not retrieved after LLM updates caused embedding drift. Scheduled re-embedding mitigated this.
- **p99 latency spikes**: Under concurrent write-read load, p99 climbed to 1.2s. Separating read and write locks on the KV store brought this back under control.

6 VI. AREAS FOR IMPROVEMENT

Every system we build reveals what we should have done differently. For this one:

- **Training data breadth**: Our Neural Engine was trained on relatively clean, well-structured text. Real-world data is messier. Expanding training to noisier corpora—forum posts, informal reports—would improve robustness.
- **Paraphrase diversity**: The model sometimes fails to recognise that two differently phrased questions are asking the same thing. Adding paraphrase augmentation during training would close this gap.
- **Smarter indexing**: For very large corpora, HNSW parameters will need tuning and sharding strategies will need to be in place before hitting the 1M+ document threshold.
- **Faster reranking**: The cross-encoder adds meaningful latency. A lighter approximation that preserves most of the quality improvement would be worth exploring.

7 VII. CONCLUSION AND FUTURE SCOPE

7.1 A. Conclusion

We started with a practical problem and built toward a practical solution. The result is a hybrid neural database that lets users query their data in plain language—across both unstructured knowledge and structured SQL databases—without managing schemas, writing query strings, or retraining models when data changes.

The numbers support the approach: 100% accuracy on simple retrieval, 92.25% on complex multi-entity queries, 25–68ms retrieval times, and a 62% latency improvement over full LLM re-query methods. Compared to NeuralDB and vanilla RAG, our system consistently led on precision, edit retention, and query robustness.

But the more interesting finding is qualitative: the gating mechanism genuinely prevented edited facts from corrupting unrelated queries, and the CoT-trained T2SQL generator handled schema variations that stumped zero-shot baselines. These are not just benchmark wins—they reflect properties that matter in production.

7.2 B. Future Scope

1. **Reasoning over data**: Move beyond fact retrieval toward inferring answers from combinations of stored facts.
2. **Multilingual support**: Extend to non-English queries, which represent the majority of global users.
3. **External knowledge graphs**: Integrate with Wikidata and domain-specific ontologies for richer grounding.

4. **Explainability**: Surface reasoning traces so users understand why a particular answer was returned.

5. **Edge deployment**: Fine-tune compact models for deployment in latency-sensitive or offline environments.

6. **Healthcare and legal domains**: These sectors have enormous volumes of unstructured text and a strong need for accurate retrieval without complex query interfaces.

References

[1] H. Su et al., "Neural knowledge base: A survey," *arXiv preprint arXiv:1704.03142*, 2017.

[2] M. Seo et al., "Neural semantic schema for entity search," *arXiv preprint arXiv:1903.08800*, 2019.

[3] T. Brown et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[4] A. Ramesh et al., "Few-shot learning for database querying with large language models," *arXiv preprint arXiv:2103.15725*, 2021.

[5] T. Yu et al., "Spider: A large-scale dataset for text-to-SQL," in *Proc. EMNLP*, pp. 3911–3921, 2018.

[6] Y. Xu et al., "Powerful neural query rewriting for text-to-SQL," *arXiv preprint arXiv:1808.08778*, 2018.

[7] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.

[8] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[9] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017.

[10] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, pp. 4171–4186, 2019.

[11] J. Johnson, M. Douze, and H. Jegou, "FAISS: A library for efficient similarity search," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[12] Y. Malkov and D. Yashunin, "Efficient and robust approximate nearest neighbour search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–835, 2020.

[13] PostgreSQL Global Development Group, "PostgreSQL 15," 2024. [Online]. Available: https://www.postgresql.org